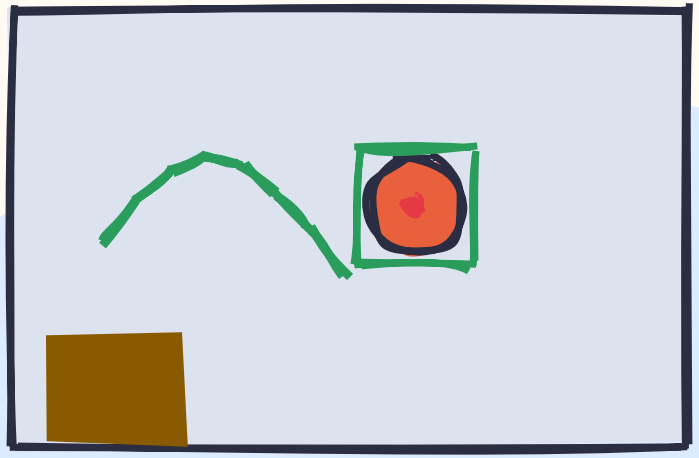
Tech Leaders · Class 17

Computer Vision with OpenCV: Colour Tracking

Teach a camera to see, find and follow.



Arrays | HSV masks | Contours | Robustness

Did you know?
Before deep learning, colour tracking powered early robot football, and it still runs on tiny robots because it's fast and simple.

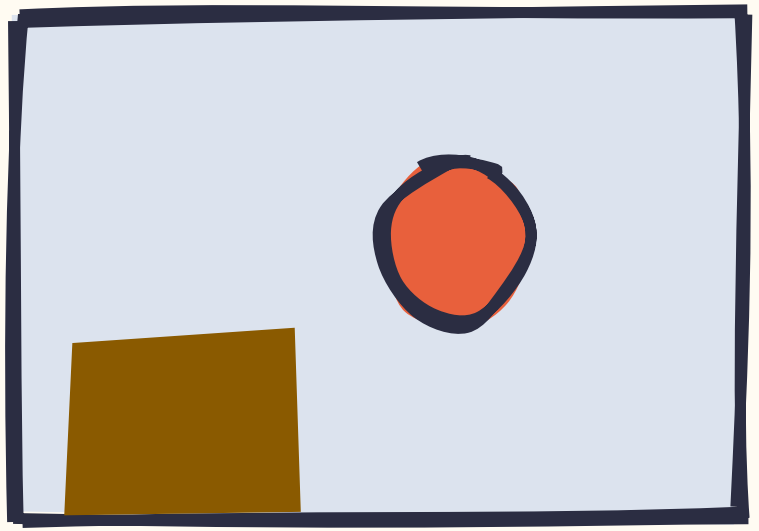
LEARN

Let's think...

What does a camera actually 'see'?

Millions of numbers

You see a ball. A computer sees **921 600 numbers** (640 × 480 × 3) changing 30 times a second. Somewhere in that sea of numbers is the ball. How do you write rules that find it, every frame, in any light?



It just knows?

Use AI?

Write the rules!

Today: you write a **Python + OpenCV** colour tracker that finds an object, marks its centre and draws its path in real time.

LEARN

Learning objectives

By the end of this class I can...

1	**Images are arrays:** treat an image as a **NumPy array** (height × width × 3, BGR, 0 to 255)	**Proof:** I can read and change one pixel's colour in code
2	**HSV thresholding:** convert to **HSV** and build a **mask** for one colour with inRange()	**Proof:** my mask shows only the target object in white
3	**Find and follow:** find **contours**, the biggest object's **centroid** and draw a tracking trail	**Proof:** a dot follows my ball live on screen
4	**Make it robust:** reduce **noise** with blur and morphology, measure **FPS**, and explain limits	**Proof:** my tracker works under two lighting conditions

LEARN

Objective 1 · Images are arrays · video

Video time!



How Computer Vision Works

CodeAI · about 6 minutes · youtu.be/2hXG8v8p0KM

Watch for...

1. How does a computer turn an image into **numbers**?
2. What are **features**, and why do they matter?
3. Where is computer vision used **today**?

Scan to watch

After the video: What would be the hardest thing for a computer to 'see' in our classroom? Why?

LEARN

Objective 1 · Images are arrays

An image is a 3D array

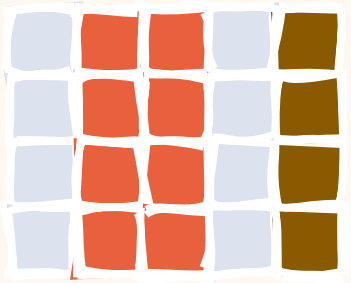
The idea

OpenCV loads a frame as a **NumPy array** with shape **(480, 640, 3)**: rows (y), columns (x), and 3 channels in **BGR** order (not RGB!). Each value is **0 to 255**.

img[100, 200] → [B, G, R] of the pixel at x = 200, y = 100.
img[100, 200] = (0, 0, 255) paints it red.

Vision = maths on arrays.

Example: a spreadsheet of colours



[60, 96, 232]

B G R

Imagine a spreadsheet 640 columns wide and 480 rows tall; every cell holds three numbers. Image processing is just very fast spreadsheet maths.

LEARN

Objective 2 · HSV thresholding

HSV: separate colour from brightness

The idea

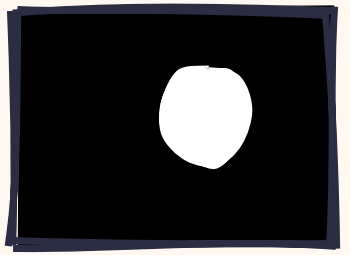
In BGR, a shadow changes all three numbers. In **HSV**:
Hue = which colour (OpenCV: 0 to 179)
Saturation = how strong (0 to 255)
Value = how bright (0 to 255)

An orange ball keeps its **hue** (≈ 5 to 20) in sun or shade.
cv2.inRange(hsv, lower, upper) → a **mask**: white where the colour matches, black elsewhere.

Example: sorting by paint name



Hue = colour angle



Sorting clothes by 'red', 'blue' (hue) works in a dim or bright room. Sorting by exact RGB numbers fails the moment a cloud covers the sun.

LEARN

Objective 3 · Find and follow

Contours and centroids

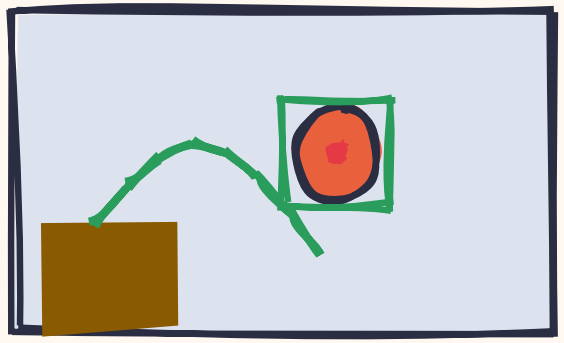
The idea

cv2.findContours(mask, ...) traces the outlines of every white blob. Keep the **largest** by area (likely our object).

Moments give the centre of mass:
cx = M['m10'] / M['m00'], **cy = M['m01'] / M['m00']**
(m00 is the area).

Store recent centres in a **deque** and draw lines between them: a trail.

Example: finding the middle of a shape



Cut a shape from card and balance it on a pencil. The balance point is the centroid. Moments compute it from pixels.

LEARN

Objective 4 · Make it robust

Noise, speed and limits

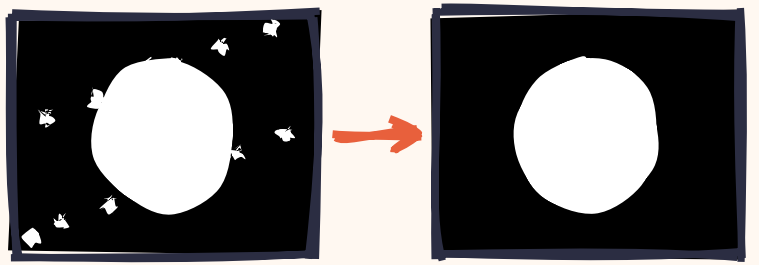
The idea

Masks are speckly. Fix with **GaussianBlur** before thresholding and **erode** → **dilate** (an 'opening') after.

Speed: smaller frames = faster. Measure **FPS** with time.time(). 640×480 → 320×240 is 4× fewer pixels.

Limits: same-colour background, coloured lighting, motion blur. That's why modern systems add **deep learning**, which you'll meet later.

Example: cleaning a photo with an eraser



Rub out tiny dots (erode), then thicken what's left back to size (dilate). Small specks vanish; the real shape survives.

LEARN

Real world

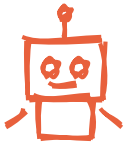
Computer vision at work



Fruit sorting
Factories sort mangoes and tomatoes by colour at high speed



Lane detection
Cars find white and yellow lane lines



Robot football
Small robots track an orange ball by colour



Medical imaging
Software highlights regions in scans for doctors

Where could a simple colour tracker solve a real problem at our school?

Classic + AI
Many real systems still use classic CV like this for speed, then AI only where it's needed.

LEARN

Quick quiz

Vision check

Question	Answer
1. Shape of a 640×480 colour image array?	
2. OpenCV's channel order?	
3. Why HSV instead of BGR for tracking?	
4. Centroid x from moments?	
5. Two ways to remove mask speckles?	

Answers

Cover! 1: (480, 640, 3) · 2: BGR · 3: hue stays stable when brightness changes · 4: m10 / m00 · 5: blur first, erode + dilate after

Part 2 · Make it

Now let's put it to work!

You know the ideas. Time to play, build and test. Use these slides while you work:

1. Human pixel grid
2. Our vision lab
3. Find your colour's HSV range
4. Track: mask → contour → centroid → trail
5. Stress-test your tracker
6. Tracker report
7. Vision engineer challenges

MAKE

Class game

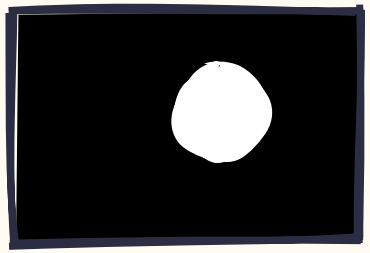
Human pixel grid

How to play

1. Floor grid of 8 × 6 squares (masking tape). Students hold coloured cards = pixels.
2. The 'algorithm' (one student) calls rules: 'Everyone holding ORANGE, raise your card!' That's the mask.
3. Find the biggest connected group of raised cards: that's the contour. Its middle student = centroid.
4. Add 'noise': 2 random students raise cards by mistake. How do we ignore them?

Then discuss

Which rule removed the noise: 'keep only the biggest group' or 'ignore groups smaller than 3'?



Big idea

Every vision step is a simple rule applied to every pixel.

MAKE

Build it

Our vision lab

Python + OpenCV
pip install opencv-python numpy

Webcam
Laptop camera works

Bright objects
Orange ball, green cap, blue marker

Plain background
A sheet of card helps

Lamp
Test different lighting

HSV picker script
Find your object's range

Run Python from Thonny or VS Code. Press q in the video window to quit; otherwise the camera stays locked.

Camera privacy
Only record yourself and people who agree. Don't save frames with classmates' faces without permission.

MAKE

Objective 2 · HSV thresholding · Build step 1 of 3

Find your colour's HSV range

```python
import cv2, numpy as np

cap = cv2.VideoCapture(0)                        # first webcam
cv2.namedWindow("controls")
for n, v in [("Hmin", 5), ("Hmax", 20), ("Smin", 120), ("Vmin", 80)]:
    cv2.createTrackbar(n, "controls", v, 255 if n[0] != "H" else 179, lambda x: None)

while True:
    ok, frame = cap.read()
    if not ok: break
    frame = cv2.resize(frame, (640, 480))
    blur = cv2.GaussianBlur(frame, (11, 11), 0)
    hsv = cv2.cvtColor(blur, cv2.COLOR_BGR2HSV)

    g = lambda n: cv2.getTrackbarPos(n, "controls")
    lower = np.array([g("Hmin"), g("Smin"), g("Vmin")])
    upper = np.array([g("Hmax"), 255, 255])
    mask = cv2.inRange(hsv, lower, upper)

    cv2.imshow("frame", frame); cv2.imshow("mask", mask)
    if cv2.waitKey(1) & 0xFF == ord("q"): break
cap.release(); cv2.destroyAllWindows()
```

Tune

Hold your object up. Slide H min/max until only the object is white in the mask.

Write down

Note your 4 numbers: you'll hard-code them next.

Red?

Red wraps around 0/179: you need two ranges joined with cv2.bitwise_or.

MAKE

Objective 3 · Find and follow · Build step 2 of 3

Track: mask → contour → centroid → trail

```python
from collections import deque
import time
LOWER, UPPER = np.array([5, 120, 80]), np.array([20, 255, 255])  # yours!
trail = deque(maxlen=30); t0, frames = time.time(), 0
kernel = np.ones((5, 5), np.uint8)

while True:
    ok, frame = cap.read()
    if not ok: break
    hsv = cv2.cvtColor(cv2.GaussianBlur(frame, (11, 11), 0), cv2.COLOR_BGR2HSV)
    mask = cv2.inRange(hsv, LOWER, UPPER)
    mask = cv2.dilate(cv2.erode(mask, kernel, iterations=2), kernel, iterations=2)

    contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    if contours:
        c = max(contours, key=cv2.contourArea)
        M = cv2.moments(c)
        if M["m00"] > 300:                          # ignore tiny blobs
            cx, cy = int(M["m10"] / M["m00"]), int(M["m01"] / M["m00"])
            x, y, w, h = cv2.boundingRect(c)
            cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 255, 0), 2)
            cv2.circle(frame, (cx, cy), 6, (0, 0, 255), -1)
            trail.appendleft((cx, cy))
    for i in range(1, len(trail)):
        cv2.line(frame, trail[i - 1], trail[i], (0, 255, 0), max(1, 8 - i // 4))

    frames += 1
    cv2.putText(frame, f"{frames / (time.time() - t0):.0f} FPS", (10, 30), 0, 1, (255, 255, 255), 2)
    cv2.imshow("tracker", frame)
    if cv2.waitKey(1) & 0xFF == ord("q"): break
```

max()

The largest contour is usually the object, not noise.

Opening

erode then dilate removes speckles but keeps the ball's size.

FPS

Watch the number. What happens if you resize to 320×240?

MAKE

Objective 4 · Build step 3 of 3

Stress-test your tracker

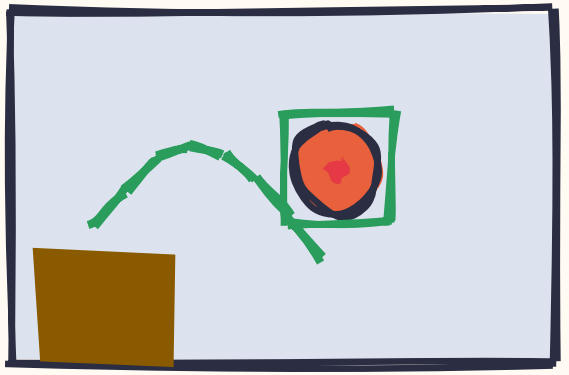
a Test under **2 lighting conditions** (lamp on/off, near window). Re-tune if needed.

b Measure FPS at 640×480 and at 320×240. Record both.

c Put a **same-colour object** in the background. What breaks? How could you fix it (area limits, region of interest)?

d Move fast: motion blur. Does the trail break?

e Print (cx, cy) every frame: next class these numbers will steer a robot!



Check: Your tracker follows the object in 2 lighting conditions and you know its FPS.

MAKE

Test like an engineer

Tracker report

Test	Result	Notes
HSV range used		
Works in light A / B		
FPS at 640×480		FPS
FPS at 320×240		FPS
Failure case found + fix		

Debug checklist

Camera won't open? Another app (Zoom, browser) is using it; try VideoCapture(1).
Mask all black? Your range is wrong: re-tune with trackbars.
Crash on m00 = 0? Keep the 'if M["m00"] > 300' check.

Good engineers report where their system FAILS, not just where it works.

MAKE

Level up!

Vision engineer challenges

Choose your challenge



Two colours: track a red and a blue object at the same time, with separate trails.



Air drawing: draw on a canvas with your object like a pen (virtual whiteboard).



Speed meter: convert pixel movement per second into an approximate speed.



Game: control a Pygame paddle with your tracked object.

Think like an engineer

Why might colour tracking fail in a factory with orange walls? What would you change?

Boss level

Estimate distance from the object's apparent size (bigger = closer) and calibrate it with a ruler.

Objective check

Can I really do it? Prove it.

01 What is the shape of a frame array, and what order are the channels?

02 Show your mask. Why did you use HSV and what range?

03 Show the centroid and trail following your object.

04 What FPS did you get, and name one failure case and your fix.

Rate yourself

Colour one circle each. Red: not yet. Amber: with help. Green: I can explain it to a friend.

Teacher: Vision demo: each team tracks a mystery object chosen by the teacher. Who tunes fastest?

Show what you know

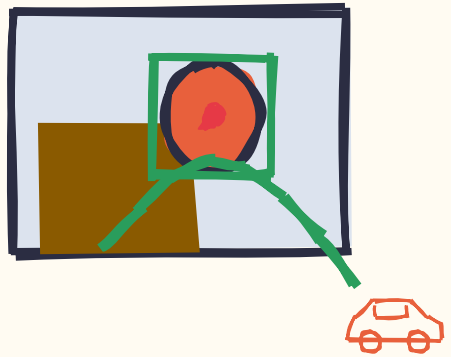
What did we learn?

- [x] Images are NumPy arrays: (h, w, 3), BGR, 0 to 255
- [x] HSV separates colour (hue) from brightness
- [x] inRange → mask; findContours → blobs
- [x] Moments give the centroid; a deque makes a trail
- [x] Blur + opening reduce noise; smaller frames run faster

Exit ticket: why does hue stay steadier than BGR values when a shadow falls on the object?

At home

Find 3 places where machines 'see' colour: traffic lights cameras, fruit sorters, phone apps.



Next class: Vision-Guided Robot Car!